UNIVERSIDAD SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
CARRERA DE CIENCIAS Y SISTEMAS
SEGUNDO SEMESTRE 2026
MANEJO E IMPLEMENTACIÓN DE ARCHIVOS
ING. BRYAN MISAEL MONZON FUENTES

MANUAL TÉCNICO PROYECTO 2

(ATENEO)

Erikson Denilson Orozco Monterroso 202131770

Quetzaltenango, 24 de Octubre de 2026

Índice

Documentación del modelo documental 3
1. Descripción general 3
2. Elementos del modelo documental 3
3. Jerarquía de carpetas (modelo de ancestros) 3
4. Estructura del documento almacenado 4
5. Relaciones entre colecciones 4
6. Ciclo de vida de un documento 5
7. Índices y consideraciones de rendimiento 5
Manual técnico 6
1. Introducción 6
2. Objetivos del Sistema 8
3. Arquitectura del Software 8
4. Modelo de Datos 10
5. Seguridad y Control de Acceso 12
6. Base de Datos y Seed 13
7. Requisitos, Compilación y Ejecución 14
8. API 16
Casos de uso 20
1. Actores 20
2. Catálogo de casos de uso 20
3. Diagrama de casos de uso 21
Diagrama de secuencia 22
1. Escenario: carga de un archivo 22
2. Participantes 22
3. Secuencia de mensajes 22
4. Diagrama de secuencia 23
Diagrama de despliegue 24
1. Nodos y componentes 24
2. Comunicación entre nodos 24
3. Diagrama de despliegue 25

Documentación del modelo documental

1. Descripción general

El modelo documental de Ateneo define cómo se representan, relacionan y almacenan los documentos institucionales. Es un modelo híbrido: la información descriptiva (metadatos, jerarquía, permisos y trazabilidad) se guarda en colecciones BSON de MongoDB, mientras que el contenido binario de cada archivo se almacena fragmentado en GridFS. Un documento se compone por tanto de dos piezas enlazadas por el campo gridFsId: su registro de metadatos en filedocs y su contenido en contents.files / contents.chunks.

2. Elementos del modelo documental

Elemento	Colección	Descripción
Usuario	users	Propietario de carpetas y archivos. Define el rol (admin o user) y acumula la cuota consumida en storageUsedBytes.
Carpeta	folders	Contenedor jerárquico. Se relaciona con su propietario (ownerId), su carpeta padre (parentId) y la lista completa de sus ancestros (ancestors).
Archivo (metadatos)	filedocs	Descripción del documento: tamaño, etiquetas (tags), extensión (txt o md) y referencia al contenido (gridFsId).
Archivo (contenido)	contents.files / contents.chunks	Contenido binario fragmentado en GridFS (bucket contents). Se lee y escribe siempre mediante streams.
Compartición	shares	Concede a un usuario beneficiario (sharedWithId) acceso a un archivo o carpeta con un nivel de permiso: read, comment o edit.
Actividad	activities	Bitácora inmutable de operaciones sobre los documentos.

3. Jerarquía de carpetas (modelo de ancestros)

Las carpetas forman un árbol de profundidad arbitraria. Además de parentId (padre directo), cada carpeta guarda en ancestors el arreglo ordenado de todos los identificadores de carpetas que la anteceden hasta la raíz. Este patrón (materialized ancestors) permite:

- Obtener todo el subárbol de una carpeta con una sola consulta indexada sobre ancestors, sin recursividad.

- Enviar una carpeta y todos sus descendientes a la papelera en una única operación.
- Validar movimientos: una carpeta no puede moverse dentro de sí misma ni de uno de sus descendientes.
- Heredar permisos de compartición desde cualquier carpeta ancestro.

Ejemplo ilustrativo de un árbol y sus ancestros:

Carpeta	parentId	ancestors
Docencia	null (raíz)	[ ]
Docencia / 2026	Docencia	[Docencia]
Docencia / 2026 / Planificaciones	Docencia / 2026	[Docencia, Docencia/2026]

Regla de unicidad: no pueden coexistir dos elementos con el mismo nombre dentro de la misma ubicación (misma carpeta padre y mismo propietario).

4. Estructura del documento almacenado

Ejemplo ilustrativo de la representación lógica de un archivo. Los campos exactos deben contrastarse con src/files/schemas/file.schema.ts.

```
// colección filedocs
{
  "_id": "ObjectId(...)",
  "name": "informe.md",
  "ownerId": "ObjectId(users)",
  "folderId": "ObjectId(folders)",
  "type": "md",                  // txt | md
  "size": 2048,                  // bytes
  "tags": ["informe", "2026"],
  "gridFsId": "ObjectId(contents.files)",
  "deletedAt": null,             // borrado lógico
  "trashRootId": null            // raíz de la eliminación
}
```

5. Relaciones entre colecciones

Origen	Relación	Destino	Campo
users	1 : N	folders	ownerId
folders	1 : N	folders (subcarpetas)	parentId / ancestors
folders	1 : N	filedocs	carpeta contenedora

Origen	Relación	Destino	Campo
users	1 : N	filedocs	propietario
filedocs	1 : 1	contents.files	gridFsId
contents.files	1 : N	contents.chunks	files_id
users	1 : N	shares	sharedWithId (beneficiario)
filedocs / folders	1 : N	shares	recurso (file o folder)
users	1 : N	activities	autor de la acción

6. Ciclo de vida de un documento

1. Carga: el archivo (.txt o .md) llega por multipart y se escribe en GridFS mediante openUploadStream; se crea el registro en filedocs y se incrementa storageUsedBytes del propietario.
2. Consulta: se previsualiza en texto crudo (/raw) o se descarga en streaming (/download) con openDownloadStream, sin copias temporales en disco.
3. Organización: el archivo puede renombrar, moverse a otra carpeta o etiquetarse; cada acción queda registrada en activities.
4. Compartición: el propietario concede acceso a otros usuarios con permiso read, comment o edit, y puede modificarlo o revocar.
5. Eliminación lógica: el archivo se marca con deletedAt y trashRootId y se excluye de las consultas normales mediante índices parciales.
6. Restauración: se limpian las marcas de borrado de todos los elementos que comparten el mismo trashRootId, de forma consistente en cascada.
7. Purga definitiva: se eliminan físicamente los chunks de GridFS y los metadatos, y se libera la cuota del usuario.

7. Índices y consideraciones de rendimiento

- Índices compuestos para listar el contenido de una carpeta por propietario y ubicación.
- Índice multikey sobre ancestors para consultas de subárboles.
- Índices parciales que excluyen los elementos con borrado lógico de las consultas habituales.
- Índices de texto para la búsqueda de archivos por término, junto con filtros por tipo y etiquetas.
- GridFS evita cargar archivos completos en memoria: el contenido viaja siempre como flujo.

Manual técnico

1. Introducción

Ateneo es un sistema de gestión documental (SGD) y almacenamiento distribuido desarrollado para la organización, catalogación, consulta y compartición colaborativa de archivos y carpetas institucionales. Está construido como una API REST en TypeScript sobre NestJS y Angular.js, respaldada por MongoDB con ReplicaSet (rs0) y por MongoDB GridFS para el almacenamiento binario de los documentos.

El sistema centraliza la administración y persistencia documental de la plataforma Ateneo, garantizando alta disponibilidad, consistencia en la jerarquía de directorios, control de acceso basado en roles (RBAC), descarga eficiente mediante flujos de streaming binario y trazabilidad exhaustiva de las operaciones de usuario.

Este documento constituye el manual técnico del proyecto y está dirigido a desarrolladores, auditores de código y personal técnico responsable de mantener, extender o evaluar la aplicación.

1.1 Alcance

El manual cubre los siguientes aspectos del sistema:

- Arquitectura modular en capas y organización de paquetes por dominio funcional (Package by Feature).
- Modelo de datos: usuarios, carpetas, archivos, compartición, actividad y almacenamiento GridFS.
- Motor de persistencia (MongoDB, Mongoose y GridFS), índices y estrategia de borrado lógico.
- Autenticación sin estado con JWT, control de acceso por roles (RBAC) y guardianes de ruta.
- Reglas de negocio: jerarquía de carpetas con ancestros indexados, permisos de compartición, papelera con trashRootId, cálculo de cuotas y auditoría.
- Módulos funcionales: autenticación, usuarios, carpetas, archivos, compartición, papelera, actividad y reportes.
- Requisitos, configuración, compilación, ejecución, seed de datos y documentación de la API.

1.2 Tecnologías utilizadas

Componente	Tecnología	Versión / Detalle	Propósito
Lenguaje	TypeScript	6.0+	Tipado estático y desarrollo de la aplicación

Componente	Tecnología	Versión / Detalle	Propósito
Entorno de ejecución	Node.js	LTS (>= 20.x)	Motor de ejecución del servidor
Framework Fronted	Angular.js	21 LTS	Arquitectura modular de maquetado
Framework base	NestJS	12.0+	Arquitectura modular, inyección de dependencias y controladores
Base de datos	MongoDB	8.x (ReplicaSet rs0)	Base NoSQL con soporte de transacciones
ODM / Modelado	Mongoose / @nestjs/mongoose	9.1+ / 12.0+	Mapeo de objetos BSON, esquemas, índices y validación
Almacenamiento binario	MongoDB GridFS	Nativo (bucket contents)	Almacenamiento y streaming de archivos en bloques
Seguridad y cifrado	Bcrypt	6.0+	Hash seguro de contraseñas con factor de coste
Autenticación	Passport JWT / @nestjs/jwt	0.7+ / 12.0+	Generación, firma y verificación de tokens de sesión
Validación de entorno	Joi	18.2+	Validación estricta de variables de entorno al arranque
Validación de DTOs	class-validator / class-transformer	0.15+ / 0.5+	Validación declarativa de cargas útiles HTTP
Documentación API	Scalar API Reference / Swagger	1.2+ / 12.0+	Especificación OpenAPI y portal interactivo
Contenedores	Docker / Docker Compose	Compose v2	Orquestación del cluster MongoDB y Mongo Express
Gestor de dependencias	PNPM	9.x o superior	Instalación y bloqueo de dependencias
Gestor de versiones	Git / GitHub	—	Control de versiones y alojamiento del repositorio

2. Arquitectura del Software

El backend implementa una arquitectura modular en capas (Modular Architecture) organizada por dominios funcionales (Package by Feature). Esta organización desacopla los controladores

de transporte, los servicios de reglas de negocio, los esquemas de persistencia y la capa de almacenamiento en GridFS. Los clientes se comunican con el sistema mediante una API REST versionada bajo el prefijo /api/v1.

2.1 Descripción de las capas

Capa / Paquete	Responsabilidad
controller	Controladores REST decorados con @Controller que exponen los endpoints HTTP bajo /api/v1, gestionan interceptores multipart (FileInterceptor), transforman parámetros y exponen la especificación OpenAPI.
auth/guards, auth/decorators	Guardián global JwtAuthGuard con soporte de rutas públicas mediante @Public(); validación de privilegios con @Roles(...) y RolesGuard; extracción tipada del usuario con @CurrentUser().
service	Servicios inyectables con las reglas de negocio: validación de nombres únicos por ubicación, resolución de ancestros, orquestación de permisos, cálculo de cuotas y registro de auditoría.
dto	Clases de transferencia de datos validadas en tiempo de ejecución con class-validator y transformadas con class-transformer a través del ValidationPipe global.
schemas	Modelos y esquemas de Mongoose con índices compuestos, índices parciales para borrado lógico e índices de texto para búsqueda rápida.
gridfs.service.ts	Abstracción del bucket GridFS de MongoDB (contents) mediante flujos de lectura (openDownloadStream) y escritura (openUploadStream).
activity, reports	Registro cronológico inmutable de eventos del sistema y canalizaciones de agregación para métricas de almacenamiento y tipos de archivo.
config	Validación estricta de variables de entorno con Joi (env.validation.ts).
database	Módulo de seed para limpieza y carga masiva de datos funcionales de demostración.

2.2 Flujo general de una operación

Toda solicitud HTTP sigue el mismo flujo entre capas:

1. La solicitud entrante pasa por el ValidationPipe global, donde se sanitizan y validan las propiedades del cuerpo y los parámetros de consulta.
2. El guardián JwtAuthGuard valida el token Bearer del encabezado Authorization; si el endpoint está restringido por rol, RolesGuard corrobora los privilegios del usuario.
3. El controlador recibe la petición y delega la operación al servicio correspondiente.

4. El servicio aplica las validaciones de negocio, interactúa con Mongoose para consultar o modificar documentos BSON y, en caso de archivos, invoca a GridFsService para gestionar los chunks binarios.
5. Cada operación relevante (subida, movimiento, compartición o eliminación) registra una entrada en la colección de actividades a través de ActivityService.
6. En operaciones sobre archivos, el servicio actualiza de manera atómica el campo storageUsedBytes del usuario propietario.
7. El controlador retorna la respuesta estructurada en JSON o un flujo binario en streaming (StreamableFile).

3.3 Estructura del proyecto Backend

```
ateneo-back/
├── compose.yml             # Servicios Docker (MongoDB ReplicaSet y Mongo Express)
├── mongo-docker/           # Dockerfile y keyfile de MongoDB
├── package.json            # Dependencias y scripts
├── pnpm-lock.yaml          # Bloqueo de versiones
├── src/
│   ├── main.ts             # Entrada: Scalar, Swagger y CORS
│   ├── app.module.ts       # Módulo raíz
│   ├── app.controller.ts   # Comprobación de estado
│   ├── app.service.ts
│   ├── activity/           # Auditoría y trazabilidad
│   ├── auth/               # Autenticación y autorización
│   │   ├── decorators/     # @Public, @Roles, @CurrentUser
│   │   ├── dto/  guards/  strategies/
│   ├── config/             # env.validation.ts (Joi)
│   ├── database/           # seed.module / seed.service / seed.ts
│   ├── files/              # Archivos, streaming y GridFS
│   ├── folders/            # Carpetas y jerarquía documental
│   ├── reports/            # Analítica y reportes
│   ├── shares/             # Compartición colaborativa
│   ├── trash/              # Papelera y purga
│   └── user/               # Administración de usuarios
├── test/                   # Pruebas automatizadas
├── tsconfig.json
└── vitest.config.ts
```

Cada módulo de dominio contiene su propio controller, module y service; los módulos con persistencia incluyen además las carpetas dto/ y schemas/.

3. Modelo de Datos

El sistema utiliza un almacenamiento híbrido: los metadatos se guardan en colecciones BSON estándar de MongoDB y el contenido binario de los archivos se aloja en las colecciones nativas de GridFS (contents.files y contents.chunks). Esta sección describe las colecciones con los atributos confirmados en la documentación del proyecto; el detalle completo de campos y tipos se encuentra en los archivos *.schema.ts de cada módulo y en la especificación OpenAPI.

3.1 Entidades de seguridad (Usuario)

Colección	Descripción
users	Cuentas de usuario con credenciales protegidas mediante hash Bcrypt, rol de acceso (admin o user) y bytes de cuota consumidos (storageUsedBytes). La desactivación de usuarios se realiza mediante soft delete.

3.2 Entidades de jerarquía documental (Carpeta)

Colección	Descripción
folders	Jerarquía de directorios con referencias ownerId y parentId, arreglo indexado de ancestors para navegación y búsqueda en profundidad arbitraria, y campos de borrado lógico (deletedAt, trashRootId). Usa índices compuestos e índices parciales que excluyen los elementos en papelera.

3.3 Entidades de archivos (Metadatos y GridFS)

Colección	Descripción
filedocs	Metadatos de archivos: tamaño en bytes, arreglo de etiquetas (tags), tipo de extensión (txt o md), referencia gridFsId al contenido binario y campos de borrado lógico. Cuenta con índices de texto para búsqueda rápida.
contents.files / contents.chunks	Almacenamiento binario fragmentado nativo de MongoDB GridFS (bucket contents).

3.4 Entidades de compartición (Share)

Colección	Descripción
shares	Relaciones de compartición que vinculan un recurso (file o folder) con un beneficiario (sharedWithId) y un nivel de permiso: read (lectura), comment (comentario) o edit (edición).

3.5 Entidad de auditoría (Actividad)

Colección	Descripción
activities	Bitácora inmutable de eventos operacionales (carga, descarga, movimiento, renombrado, compartición, eliminación y restauración) con detalles contextuales, marcas de tiempo y tipo de recurso afectado.

3.6 Reglas del modelo

- **Ancestros indexados:** cada carpeta almacena la lista de sus ancestros, de modo que consultar o mover un subárbol no requiere recorridos recursivos.
- **Borrado lógico:** al enviar a la papelera se marca deletedAt y se asigna trashRootId, que identifica la raíz de la eliminación y permite restaurar en cascada de forma consistente.
- **Purga física:** la eliminación definitiva borra los chunks en GridFS y los metadatos asociados.
- **Cuota de almacenamiento:** storageUsedBytes se actualiza atómicamente en cada operación que agrega o libera bytes.

4. Seguridad y Control de Acceso

4.1 Autenticación con JWT

La autenticación es sin estado. Al iniciar sesión (POST /api/v1/auth/login) el servidor valida las credenciales contra el hash Bcrypt y emite un JSON Web Token firmado con HMAC-SHA usando JWT_SECRET (mínimo 32 caracteres). El cliente debe enviar el token en cada solicitud mediante el encabezado:

```
Authorization: Bearer <TOKEN_JWT>
```

El tiempo de vida del token se controla con la variable JWT_EXPIRES_IN (por defecto 1d).

4.2 Autorización por roles (RBAC)

- **JwtAuthGuard:** guardián global que protege todos los endpoints; las rutas de registro e inicio de sesión se exponen con @Public().
- **RolesGuard y @Roles(...):** restringen endpoints por rol. Los roles existentes son admin y user.
- **@CurrentUser():** decorador que entrega el usuario autenticado tipado al controlador.

4.3 Permisos sobre recursos compartidos

Permiso	Alcance
read	Consulta y previsualización/descarga del recurso compartido.
comment	Lectura y comentario sobre el recurso.
edit	Lectura, comentario y edición colaborativa del recurso.

5. Base de Datos y Seed

5.1 Motor y configuración

- **Motor:** MongoDB 8.x con ReplicaSet de un solo nodo (rs0), configurado para habilitar transacciones ACID de documentos cuando se requieran.
- **Despliegue:** el cluster se levanta con Docker Compose usando la imagen personalizada de mongo-docker/Dockerfile y un keyfile para autenticación interna.
- **Visor web:** Mongo Express disponible en http://localhost:8081 para inspección interactiva de los datos.

5.2 Seed de datos

El comando pnpm run seed limpia las colecciones previas y genera un ecosistema de datos listo para la demostración funcional. También puede ejecutarse automáticamente al arrancar la aplicación con SEED_ON_BOOT=true.

Cuentas de usuario inicializadas (todas con la contraseña uniforme "Password"):

Usuario	Correo electrónico	Rol	Perfil y propósito
admin	admin@ateneo.com	admin	Administrador general de plataforma y políticas
edk	edk@ateneo.com	user	Usuario principal docente e investigador
maria	maria.lopez@ateneo.com	user	Coordinadora de vinculación comunitaria y extensión
carlos	carlos.mendoza@ateneo.com	user	Investigador y autor de publicaciones científicas
user	user@ateneo.com	user	Usuario estándar para verificación de permisos

Estructura de datos generada:

- **Carpetas multinivel:** organizadas por departamentos, con subcarpetas anidadas y cálculo estricto de ancestros.
- **Archivos reales en GridFS:** documentos .md y .txt con contenido técnico, académico y administrativo que permiten descargas y previsualizaciones funcionales.
- **Recursos compartidos:** carpetas y archivos compartidos entre usuarios con permisos de lectura, comentario y edición.
- **Papelera funcional:** elementos borrados lógicamente para verificar restauración y purga permanente.
- **Bitácora de actividad:** acciones de subida, compartición y eliminación para alimentar el dashboard y la línea de tiempo.
- **Métricas de cuota:** porcentajes de almacenamiento consumido calculados para cada usuario.

6. Requisitos, Compilación y Ejecución

6.1 Requisitos del sistema

- **Node.js:** versión 20.x LTS o superior.
- **PNPM:** versión 9.x o superior (npm install -g pnpm).
- **Docker y Docker Compose:** instalados y con el servicio Docker activo.
- **Git:** para clonar el repositorio.

6.2 Clonado del repositorio

```
git clone https://github.com/EDK367/ateneo-back.git
cd ateneo-back
```

6.3 Instalación de dependencias

```
pnpm install
```

6.4 Variables de entorno

Copie la plantilla y ajuste los valores (cp .env.example .env). Las propiedades se validan con Joi en src/config/env.validation.ts.

Variable	Descripción	Oblig.	Ejemplo
PORT	Puerto HTTP del servidor	No (3000)	3000
DB_USER	Usuario administrador raíz de MongoDB	Sí	admin

Variable	Descripción	Oblig.	Ejemplo
DB_PASSWORD	Contraseña del administrador de MongoDB	Sí	admin
DB_NAME	Nombre de la base de datos principal	Sí	ateneo_db
DB_MONGO_URI	URI de conexión a MongoDB con ReplicaSet	Sí	mongodb://admin:admin@localhost:27017/?replicaSet=rs0&directConnection=true
JWT_SECRET	Clave secreta HMAC-SHA para firmar los JWT (mínimo 32 caracteres)	Sí	cadena larga y segura
JWT_EXPIRES_IN	Tiempo de vida del token de acceso	No (1d)	1d
FRONTEND_URL	Origen frontend autorizado para CORS	No	http://localhost:4200
SEED_ON_BOOT	Ejecuta el seed automáticamente al arrancar (true/false)	No (false)	false

6.5 Inicio de servicios en Docker

```
docker compose up -d
docker compose ps
```

El segundo comando permite verificar que los contenedores estén en ejecución y en estado saludable.

6.6 Inicialización de la base de datos

```
pnpm run seed
```

6.7 Ejecución de la aplicación

Modo desarrollo (con recarga automática):

```
pnpm run start:dev
```

Modo producción:

```
pnpm run build
pnpm run start:prod
```

7. API

7.1 Documentación interactiva

El proyecto expone documentación OpenAPI 3.0 mediante la interfaz Scalar:

- **Portal interactivo:** http://localhost:3000/docs
- **Especificación OpenAPI (JSON):** http://localhost:3000/api

Todos los endpoints, excepto registro e inicio de sesión, requieren el encabezado Authorization: Bearer <TOKEN_JWT>.

7.2 Auth

Método	Endpoint	Rol	Descripción
POST	/api/v1/auth/register	Público	Registro de nueva cuenta de usuario
POST	/api/v1/auth/login	Público	Inicio de sesión y obtención de token JWT
GET	/api/v1/auth/profile	Autenticado	Datos del usuario en sesión

7.3 Users

Método	Endpoint	Rol	Descripción
GET	/api/v1/user	User / Admin	Listado general de usuarios registrados
POST	/api/v1/user	Admin	Creación directa de usuario
PUT	/api/v1/user/:id	Admin	Actualización completa de datos de usuario
PATCH	/api/v1/user/:id	Admin	Actualización parcial de datos de usuario
DELETE	/api/v1/user/:id	Admin	Desactivación de usuario (soft delete)

7.4 Folders

Método	Endpoint	Rol	Descripción
POST	/api/v1/folders	Autenticado	Creación de carpeta en la raíz o subcarpeta
GET	/api/v1/folders/content	Autenticado	Contenido de una carpeta o de la raíz

Método	Endpoint	Rol	Descripción
PATCH	/api/v1/folders/:id/move	Autenticado	Traslado de carpeta a un nuevo destino
PATCH	/api/v1/folders/:id/rename	Autenticado	Renombramiento de carpeta
DELETE	/api/v1/folders/:id	Autenticado	Envío de carpeta y descendientes a papelera

7.5 Files

Método	Endpoint	Rol	Descripción
POST	/api/v1/files/upload	Autenticado	Carga multipart de archivo (.txt / .md) hacia GridFS
GET	/api/v1/files/search	Autenticado	Búsqueda por término, tipo y etiquetas
GET	/api/v1/files/recent	Autenticado	Archivos recientes del usuario
GET	/api/v1/files/:id/raw	Autenticado	Previsualización en texto crudo
GET	/api/v1/files/:id/download	Autenticado	Descarga directa en streaming binario
PATCH	/api/v1/files/:id/rename	Autenticado	Renombramiento de archivo
PATCH	/api/v1/files/:id/move	Autenticado	Movimiento a otra carpeta
PATCH	/api/v1/files/:id/tags	Autenticado	Actualización de etiquetas
DELETE	/api/v1/files/:id	Autenticado	Envío de archivo a la papelera

7.6 Shares

Método	Endpoint	Rol	Descripción
POST	/api/v1/shares	Autenticado	Compartir archivo o carpeta con un usuario
GET	/api/v1/shares/shared-with-me	Autenticado	Recursos compartidos con el usuario

Método	Endpoint	Rol	Descripción
GET	/api/v1/shares/shared-by-me	Autenticado	Recursos que el usuario ha compartido
PATCH	/api/v1/shares/:id/authorization	Autenticado	Modificar nivel de permiso (read, comment, edit)
DELETE	/api/v1/shares/:id	Autenticado	Revocar acceso a recurso compartido

7.7 Trash

Método	Endpoint	Rol	Descripción
GET	/api/v1/trash	Autenticado	Listado de elementos en papelera
POST	/api/v1/trash/restore/:id	Autenticado	Restauración de archivo o carpeta
DELETE	/api/v1/trash/purge/:id	Autenticado	Eliminación definitiva física de un elemento
DELETE	/api/v1/trash/empty	Autenticado	Vaciado completo de la papelera

7.8 Activity

Método	Endpoint	Rol	Descripción
GET	/api/v1/activity	Autenticado	Consulta cronológica de acciones y auditoría

7.9 Reports

Método	Endpoint	Rol	Descripción
GET	/api/v1/report/stats	Autenticado	Uso de cuota y desglose del usuario
GET	/api/v1/report/global	Admin	Métricas globales para administradores
GET	/api/v1/report/by-type	Autenticado	Archivos agrupados por tipo (.txt, .md)
GET	/api/v1/report/by-date-range	Autenticado	Archivos almacenados en un rango de fechas

Método	Endpoint	Rol	Descripción
GET	/api/v1/report/most-active-files	Autenticado	Archivos con más operaciones registradas
GET	/api/v1/report/top-users-files	Autenticado	Top 10 usuarios con más archivos
GET	/api/v1/report/top-users-shares	Autenticado	Usuarios con más recursos compartidos
GET	/api/v1/report/shared-by-user/:userId	Autenticado	Recursos compartidos por un usuario
GET	/api/v1/report/shared-with-user/:userId	Autenticado	Recursos recibidos por un usuario
GET	/api/v1/report/never-shared	Autenticado	Archivos que nunca han sido compartidos
GET	/api/v1/report/folders-most-files	Autenticado	Carpetas con mayor cantidad de archivos
GET	/api/v1/report/files-most-shared-users	Autenticado	Archivos compartidos con más usuarios

Diagramas
https://drive.google.com/file/d/1bsR10VXwUMu7obuuNPduyNaEYE58EGRh/view?usp=drive_link

1. **Casos de uso**

Actores

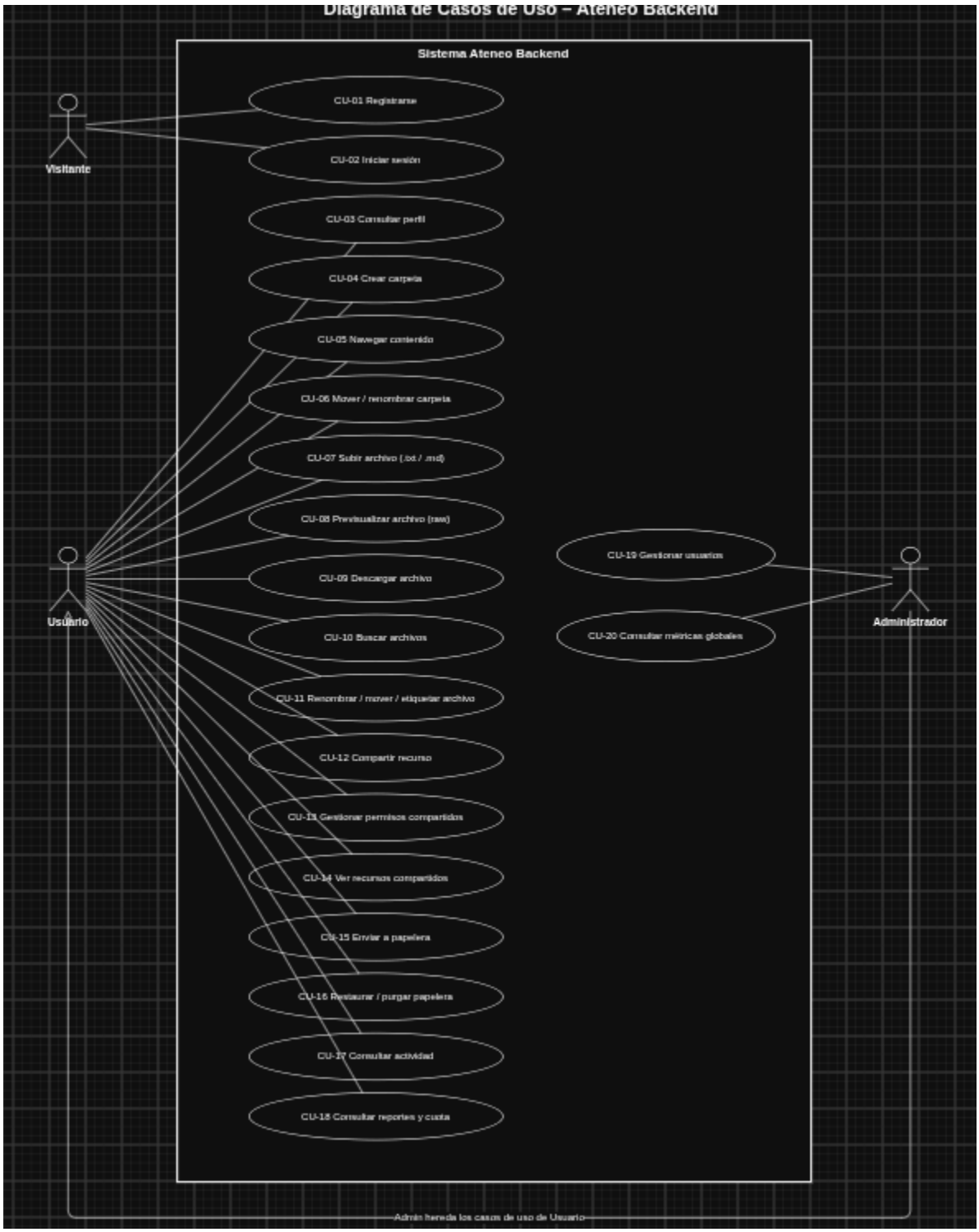
Actor	Descripción
Visitante	Persona sin sesión iniciada. Solo accede a las rutas públicas de registro e inicio de sesión.
Usuario (user)	Usuario autenticado. Gestiona sus carpetas y archivos, comparte recursos, usa la papelera y consulta su actividad y reportes.
Administrador (admin)	Hereda todos los casos de uso del usuario y además administra cuentas y consulta métricas globales.

Catálogo de casos de uso

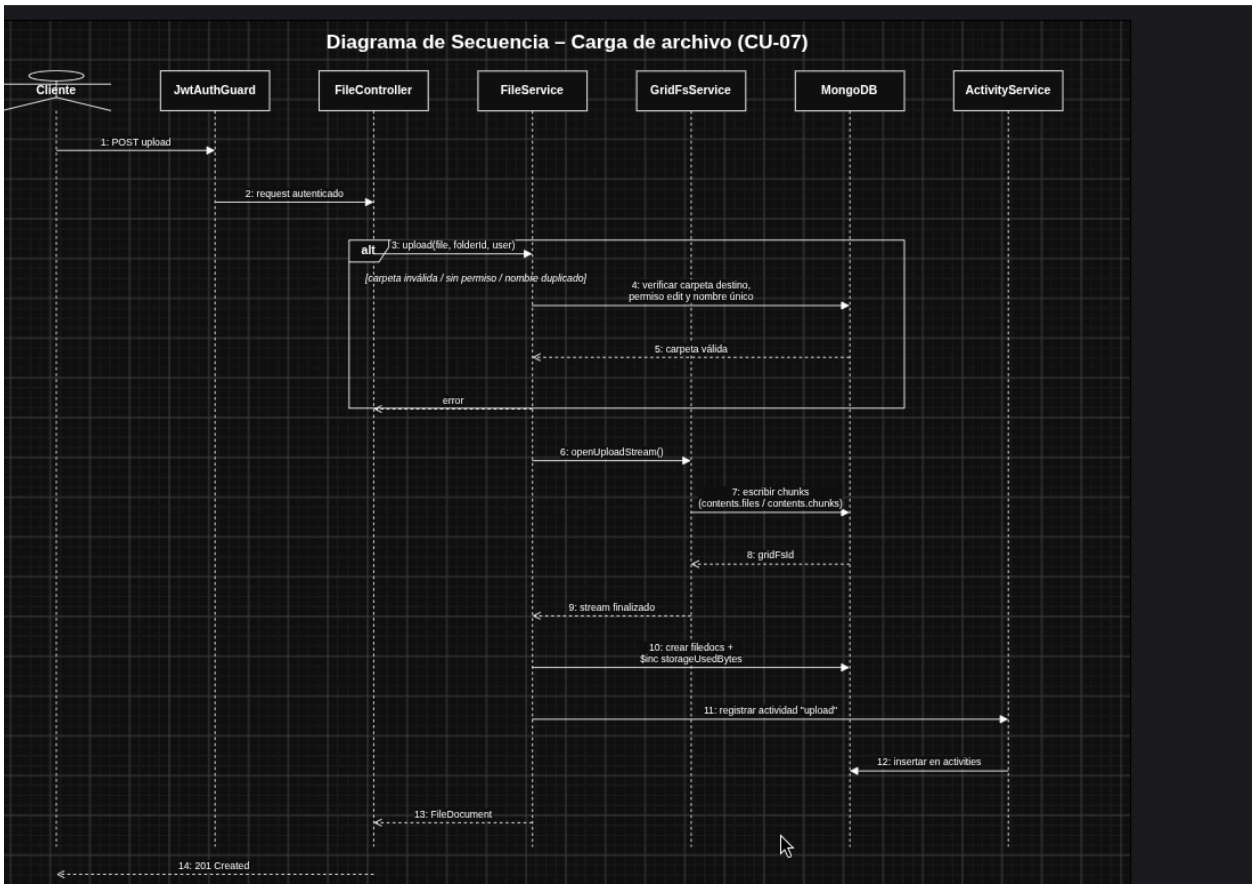
ID	Caso de uso	Actor	Descripción
CU-01	Registrarse	Visitante	Crea una nueva cuenta de usuario (POST /auth/register).
CU-02	Iniciar sesión	Visitante	Valida credenciales y entrega un token JWT (POST /auth/login).
CU-03	Consultar perfil	Usuario	Obtiene los datos del usuario en sesión.
CU-04	Crear carpeta	Usuario	Crea una carpeta en la raíz o dentro de otra carpeta.
CU-05	Navegar contenido	Usuario	Lista carpetas y archivos de una carpeta o de la raíz.
CU-06	Mover / renombrar carpeta	Usuario	Cambia el nombre o la ubicación de una carpeta, recalculando ancestros.
CU-07	Subir archivo	Usuario	Carga un archivo .txt o .md hacia GridFS y actualiza la cuota.
CU-08	Previsualizar archivo	Usuario	Muestra el contenido en texto crudo (raw).
CU-09	Descargar archivo	Usuario	Descarga el archivo mediante streaming binario.
CU-10	Buscar archivos	Usuario	Busca y filtra por término, tipo y etiquetas; consulta recientes.

ID	Caso de uso	Actor	Descripción
CU-11	Renombrar / mover / etiquetar archivo	Usuario	Modifica el nombre, la carpeta o las etiquetas de un archivo.
CU-12	Compartir recurso	Usuario	Comparte un archivo o carpeta con otro usuario con permiso read, comment o edit.
CU-13	Gestionar permisos compartidos	Usuario	Modifica el nivel de permiso o revoca el acceso a un recurso compartido.
CU-14	Ver recursos compartidos	Usuario	Lista lo compartido conmigo y lo compartido por mí.
CU-15	Enviar a papelera	Usuario	Elimina lógicamente un archivo o carpeta (con descendientes).
CU-16	Restaurar / purgar papelera	Usuario	Restaura en cascada, purga un elemento o vacía la papelera.
CU-17	Consultar actividad	Usuario	Revisa la bitácora cronológica de acciones.
CU-18	Consultar reportes y cuota	Usuario	Consulta estadísticas personales, cuota consumida y reportes por tipo, fechas y compartición.
CU-19	Gestionar usuarios	Administrador	Lista, crea, actualiza y desactiva cuentas de usuario.
CU-20	Consultar métricas globales	Administrador	Consulta las métricas globales del sistema (/report/global).

2. Diagrama de casos de uso



3. Diagrama de secuencia



4. Diagrama de despliegue

